sniffy — Proposed Project Structure

Design-only reference document. No repository code was changed to produce this plan.

Context — why this restructure

> The repo is a two-part ML pipeline: `proj/` captures live traffic with NFStream/pyshark and extracts NetFlow-style features, and `model/` trains a two-stage XGBoost classifier (stage 1: binary attack/benign, stage 2: multiclass attack type on the positives). Right now everything is flat procedural scripts with no packages (no `__init__.py` anywhere), no shared modules, and heavy copy-paste:
>
> - The 43-column NetFlow schema is hardcoded verbatim in 4 places (`model/split_csv.py`, `model/train_v1.py`, `proj/test/test_colunm.py`, and implicitly in `proj/classes/nfstram_processor.py`).
> - Feature-prep (drop cols → clean → split) is duplicated 3×; the incremental-XGBoost-train-and-metrics block is duplicated 3×.
> - `proj/test/` triplicates several NFStream plugin classes (`tcp_win.py`, `dns_values.py`) as scratch copies.
> - `model/train_v1.py` (376 lines) has real bugs traceable directly to this duplication:
>   - `prepare_features` drops lowercase column names (`"label"`) against uppercase real columns → `KeyError`.
>   - `train_stage_two` references `model` without ever initializing it in that function's scope → `NameError`.
>   - Stage 1 and stage 2 both dump to the same `xgb_attack_model.pkl`, so stage 2 silently overwrites stage 1's artifact.
>   - `calc_scale_pos_weight` in stage 2 reads the stage-1 directory instead of the segregated attack-only one.
>   - The module runs `train_stage_two(...)` unconditionally at import time — no `if __name__ == "__main__"` guard anywhere in the repo.
> - `proj/classes/nfstram_processor.py` builds a flow record dict but only `print()`s it — there's no actual persistence step from live capture to a trainable CSV.
>
> The goal is to turn ~15 scattered scripts into an installable package with a single source of truth for the schema, one reusable data-prep/training class hierarchy (fixing the bugs above *by construction*, not by patching them individually), one canonical set of capture plugins, and a real entrypoint layer — while keeping the exact same modeling approach (two-stage XGBoost, NFStream-based capture) already in place.

Target directory structure

```
sniffy/
├── README.md
├── requirements.txt            # regenerated to include nfstream, dpkt, pyshark, netifaces, matplotlib, pyarrow (currently
├── pyproject.toml              # makes `sniffy` pip-installable (-e .) so imports work from anywhere
├── .gitignore                  # fix "vevn"->".venv", add "artifacts/", "*.pcap", "*.csv" data ignores
├── sniffy/                     # the installable package (all real logic lives here)
│   ├── __init__.py
│   ├── config.py               # Paths/Hyperparams dataclasses — replaces every hardcoded /mnt/8tb_hdd/... and /home/h
│   ├── schema.py               # single COLUMNS list + DROP_COLS_STAGE1 / DROP_COLS_STAGE2 / TARGET_STAGE1 / TA
│   ├── data/
│   │   ├── __init__.py
│   │   ├── loader.py           # DatasetSplitter, DirectoryScanner classes
│   │   ├── preprocessing.py    # FeaturePreprocessor class
│   │   └── segregation.py      # AttackSegregator class (create + verify stage-2 dataset)
│   ├── models/
│   │   ├── __init__.py
│   │   ├── base_trainer.py     # XGBIncrementalTrainer base class
│   │   ├── stage1_trainer.py   # BinaryAttackTrainer(XGBIncrementalTrainer)
│   │   ├── stage2_trainer.py   # AttackTypeTrainer(XGBIncrementalTrainer)
│   │   └── predictor.py        # AttackPredictor — cascades stage1 -> stage2 for inference
│   ├── capture/
│   │   ├── __init__.py
│   │   ├── live_capture.py     # PacketCapturer (pyshark live capture wrapper)
│   │   ├── stream_processor.py # FlowStreamProcessor — builds flow_record from schema.py AND writes it to CSV
│   │   └── plugins/
│   │       ├── __init__.py
│   │       ├── dns.py          # DnsPlugin (one canonical copy, was duplicated in classes/ and test/)
│   │       ├── tcp.py          # TcpWindowPlugin (was triplicated across classes/, test/tcp_win.py, test/coc.py)
│   │       ├── icmp.py         # IcmpPlugin
│   │       ├── ttl.py          # TtlPlugin
│   │       └── packet_size.py  # PacketSizePlugin
│   └── viz/
│       ├── __init__.py
│       └── eda.py              # EDAVisualizer class (parameterized dataset path, replaces datavisuvalisation.py + orphaned
├── scripts/                    # thin CLI entrypoints, each just wires config + a class together
│   ├── split_dataset.py        # calls DatasetSplitter
│   ├── train_stage1.py         # calls BinaryAttackTrainer
│   ├── train_stage2.py         # calls AttackTypeTrainer
│   ├── capture.py              # calls PacketCapturer + FlowStreamProcessor
│   └── predict.py              # calls AttackPredictor
├── artifacts/                  # gitignored; versioned model output, e.g. artifacts/stage1/xgb_v1.pkl
└── tests/                      # placeholder for future pytest coverage of preprocessing.py/schema.py
```

Files retired / merged (documented, not deleted by this plan)

- `model/train.py`, `model/train_v1.py`, `model/test.py` → `sniffy/models/*` + `scripts/train_stage{1,2}.py`
- `model/split_csv.py` → `sniffy/data/loader.py` + `scripts/split_dataset.py`
- `model/fomate_change.py`, `model/inference.py`, `model/datavisuvalisation.py` → `sniffy/viz/eda.py`, `sniffy/models/predictor.py`

- `proj/classes/*.py` → `sniffy/capture/plugins/*.py` (one file per feature family; small unused helpers like `calc_bps` / `flags_stats` / `icmp_info` folded into the plugin that would use them)
- `proj/test/*` and `.ipynb_checkpoints/*` → deleted (scratch/duplicate content, including an untracked 71.6 MB `capture.pcap` not currently gitignored)
- `capture.py` (empty 0-line stub at repo root) → removed; real entrypoint becomes `scripts/capture.py`

Key class designs — what and why

schema.py — single source of truth

```
COLUMNS: list[str]              # the 43-column NetFlow schema
DROP_COLS_STAGE1: list[str]     # uppercase — fixes the lowercase/uppercase
KeyError bug
DROP_COLS_STAGE2: list[str]
TARGET_STAGE1 = "Label"
TARGET_STAGE2 = "Attack"
```

Why: the schema is currently copy-pasted in 4 places. One drift between copies (e.g. lowercase vs uppercase) is exactly what causes `prepare_features`'s `KeyError` today. A single constants module makes that class of bug structurally impossible.

data/preprocessing.py :: FeaturePreprocessor

```
class FeaturePreprocessor:
    def __init__(self, drop_cols, target_col, test_size): ...
    def transform(self, df) -> (X_train, X_test, y_train, y_test): ...
```

Why: replaces the 3× duplicated `prepare_features` functions (in `train.py` and twice in `train_v1.py`). Stage 1 vs stage 2 differ only in which columns get dropped and what the target is — that's now just two different constructor calls, not two copies of the same body.

data/loader.py :: DirectoryScanner, DatasetSplitter

```
class DirectoryScanner:
    @staticmethod
    def list_csv_files(directory: str) -> list[str]: ...

class DatasetSplitter:
```

```
    def __init__(self, chunksize: int, columns: list[str]): ...
    def split(self, source_csv: str, dest_dir: str) -> int: ...
```

Why: the `walk(...)` + `extend(filename)` directory-listing pattern is repeated 6× across the codebase (and is the exact bug in `model/test.py`, where the wrong variable gets extended). One correct implementation, reused everywhere.

models/base_trainer.py :: XGBIncrementalTrainer (abstract base)

```
class XGBIncrementalTrainer(ABC):
    def __init__(self, preprocessor, hyperparams, artifact_path): ...
    def fit_directory(self, directory: str) -> XGBClassifier:
        # walks CSVs via DirectoryScanner, warm-starts xgb_model across
files,
        # computes/prints metrics each iteration, dumps ONCE at the end
    def _compute_metrics(self, y_test, y_pred, y_proba) -> dict: ...
    @abstractmethod
    def scale_pos_weight_source(self) -> str: ...
```

Why this shape: the incremental-fit + metrics-printing block is duplicated 3× today (`train.py`, and twice more in `train_v1.py`). Pulling it into a base class fixes three concrete bugs by construction rather than by patching each script:

- `model` becomes `self.model`, initialized in `__init__` — the `NameError` in `train_stage_two` (where `model` was never set before being checked) can't happen.
- Each subclass supplies its own `artifact_path` — stage 2 can no longer silently overwrite stage 1's saved model file.
- `scale_pos_weight_source()` is abstract, forcing each stage to explicitly state which directory its class-imbalance ratio comes from — stage 2 currently reads the stage-1 directory by copy-paste accident; making it a required override surfaces that decision instead of hiding it.

`BinaryAttackTrainer` and `AttackTypeTrainer` subclass this with just their own hyperparameters, drop-cols/target (via the `FeaturePreprocessor` they're constructed with), and artifact path.

data/segregation.py :: AttackSegregator

```
class AttackSegregator:
    def create(self, source_dir: str, dest_dir: str) -> None: ...
    def verify(self, dest_dir: str) -> bool: ...
```

Why: merges `segrigate_sec_dataset` + `verify_sec_datset` into one cohesive class instead of two standalone functions that only make sense used together.

capture/plugins/ — one canonical NFPlugin subclass per feature family

Why: `tcp_win.py`-equivalent logic currently exists in three places (`proj/classes/tcp_win.py`, `proj/test/tcp_win.py`, `proj/test/coc.py`), and `dns_values.py` is duplicated in `proj/test/dns_inspector.py`. Several files also import `NFPlugin` in a way that accidentally shadows the real `nfstream.NFPlugin` with a same-named local module — consolidating to one file per plugin, importing correctly from `nfstream`, removes both the duplication and that fragile import.

capture/stream_processor.py :: FlowStreamProcessor

Why: the current `nfstram_processor.py` builds a flow record dict matching the schema but only `print()`s it — there is no code path today that turns a live capture into a CSV row the training pipeline can consume. This class closes that gap and builds the record from `schema.COLUMNS` instead of a hand-maintained dict.

models/predictor.py :: AttackPredictor

```
class AttackPredictor:
    def __init__(self, stage1_path: str, stage2_path: str): ...
    def predict(self, features: pd.DataFrame) -> pd.Series:
        # stage1.predict -> for rows predicted attack, run stage2.predict
for attack type
```

Why: today's `inference.py` is a one-off smoke test against a single hardcoded feature vector. This gives the two-stage cascade (binary → attack type) a real, reusable predict path.

viz/eda.py :: EDAVisualizer

Why: parameterizes `datavisuvalisation.py` (dataset path, label column) instead of hardcoding `dataset0.csv`; also replaces the two orphaned Jupyter-checkpoint notebooks that contain the same logic with no live source notebook behind them.

Notes for implementation

- `config.py` should hold every currently-hardcoded absolute path (`/mnt/8tb_hdd/harish/ds2/...`, `/home/h4r15h/dataset/sniffy/splitted/`) so the pipeline is portable across machines — this is also where the `argv` import that's currently unused in `train_v1.py` would finally get used, via `scripts/*.py` using `argparse`.
- Every `scripts/*.py` entrypoint should use `if __name__ == "__main__":` — the current unconditional `train_stage_two(...)` call at module scope in `train_v1.py` is what makes that file crash on import.
- Model artifacts should be versioned by stage + version (e.g. `artifacts/stage1/xgb_v1.pkl`), not the single shared `xgb_attack_model.pkl` filename both stages currently fight over.